MemDB2 C DLL Benchmark Results

mdb2bench, 12 November 2026 · x64 Release DLL · 1 million records · mixed workload 4 readers + 4 writers · read-only workload 1–8 readers

This report replaces MemDB2_Benchmark_Results_2026-11-11.pdf. That report's figures were correct for its build, but MemDB2 has changed since, and so has the way the runs are set up:

- **Journal flow control now depends on the data waiting, not the number of commits.** A commit is switched to flush-buffers when more than 128 MB of changesets are queued for the journal thread, instead of when more than 4,096 commits are. Checkpoints aren't counted.
- **The table is loaded once, with 8 threads,** each committing batches of 10,000 rows in READ_WRITE_SHARED transactions. Before each run the loaded, checkpointed database is copied, and the run reopens the copy, so all three runs start from identical data.
- **New read-only runs** measure the maximum read throughput with 1, 2, 4 and 8 readers and no writers.

63,564 txn/s
read-only peak, 4 READ_SHARED readers

1.3–8.8 ms
p99.99 of isolated writes, down from 14 ms–22 ms

-17%
mixed throughput with async checkpoints running, against -36% before

×2.2
faster preload of 1M rows with 8 threads (61 s–62 s against 125 s–144 s)

Summary

All figures come from an old Ivy Bridge laptop, with every call going through the C DLL. Expect modern hardware, and native Delphi use, to be considerably faster (see Caveats).

- **The long write tails were journal flow control.** With flow control based on queued bytes, p99.99 of isolated writes fell from 14 ms–22 ms to 1.28 ms–8.84 ms, and their means fell 5–18%. Read tails stayed at or below about 1 ms (127 µs–1.09 ms at p99.99).
- **Mixed-workload tails shrank by an order of magnitude.** In the read run p99.99 fell from 41 ms to 3.55 ms and the worst case from 254 ms to 7.88 ms; the 155 ms all-thread stall of the previous report is gone. Throughput rose 22% in the read run (33,349 txn/s) and was unchanged in the rws run (29,209).
- **Asynchronous checkpoints now cost much less.** Throughput falls 17% instead of 36%, and inserts/s fall 16% instead of 55%. The worst case fell from 1.35 s to 111 ms, and p99 from 9.43 ms to 1.22 ms. Checkpoints still lengthen p99.9 and beyond, to 17 ms and 41 ms.
- **Read-only throughput peaks at 4 readers, 63,564 txn/s.** 1 reader gives 29,050, 2 give 47,136. With 8 readers throughput falls 45% to 35,164, while CPU per transaction rises from 75k cycles (1 reader) to 212k, pointing at lock contention once threads outnumber the 4 physical cores.
- **READ_WRITE_SHARED readers have a ~14 ms tail even with no writers.** 8 readers in that mode match READ_SHARED throughput (34,426 txn/s) but reach 14 ms at p99.99 and 63 ms at worst, against 1.18 ms and 5.11 ms in READ_SHARED.
- **Loading with 8 threads is 2.2 times faster** (16,172–16,421 rows/s), with no concurrency clashes, but used 1,906 bytes of process memory per row, 17% more than the single-threaded load.

Test setup

Machine	Laptop, Intel Core i7-3632QM (Ivy Bridge, 2012), 2.2 GHz, 4 cores / 8 threads; invariant TSC (2.195 GHz)
Database	MemDB2 C DLL, x64 Release, assertions off, without the benchmark allocation hooks; journal flow control at 128 MB of queued changesets. All calls go through the C API shim (MemDB2CApi), not native Delphi
Data	1,000,000 rows of about 128 bytes (Id, Expiry, State, Payload blob); unique index on Id, index on Expiry. Loaded once by 8 threads, checkpointed; each run reopens a fresh copy
Transactions	Writers READ_WRITE_SHARED, lazy write journal, repeatable read isolation. Readers READ_SHARED or READ_WRITE_SHARED, and end every transaction with a rollback
Isolated phase	Each of 7 operations alone on one thread, 200,000 transactions each
Mixed phase	4 reader + 4 writer threads, unpaced, 30 s after a 5 s warm-up. Per insert: delete by ID p = 0.5, state transition p = 0.25, recycle p = 0.1; expiry sweep every 500 µs. Readers: find by ID, with find earliest every 10th read
Read-only phase	1, 2, 4 or 8 readers and no writers, unpaced, 60 s after a 10 s warm-up, against a separately loaded copy of the same data
Runs	`plain_tests.bat`: read (READ_SHARED readers), rws (READ_WRITE_SHARED readers), rws-ckpt (as rws, with back-to-back asynchronous checkpoints during the mixed phase). `read_only_tests.bat`: ro-read-1/2/4/8 and ro-rws-8

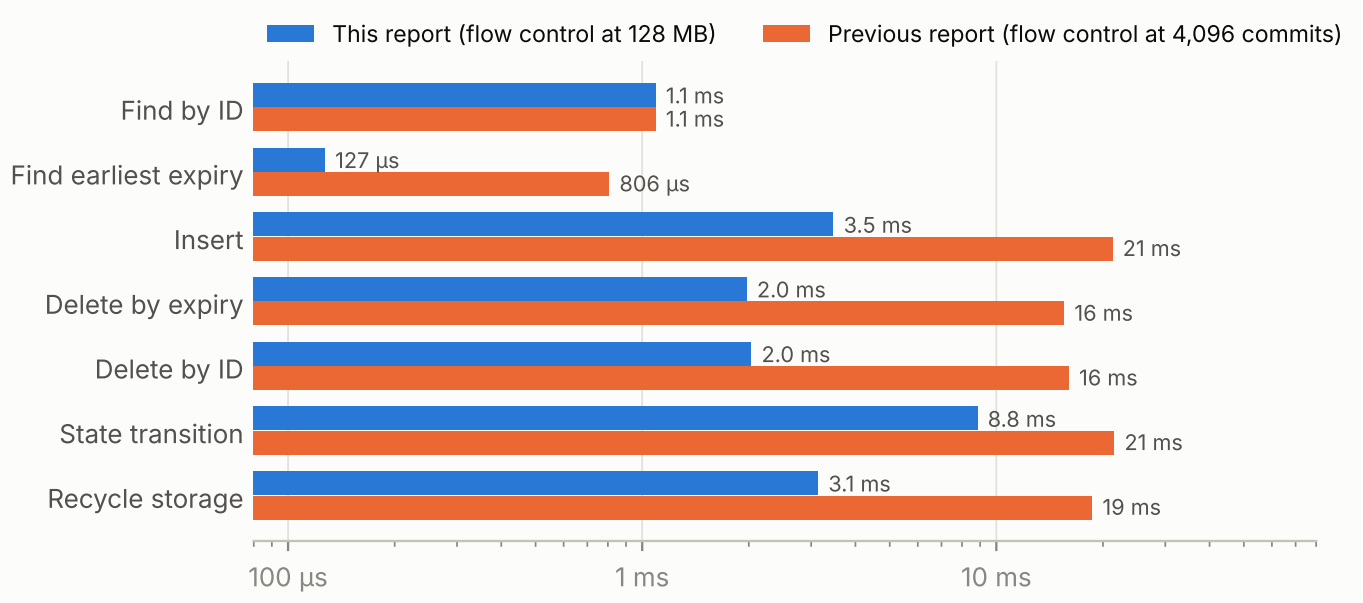
Changes since the previous report

The same three runs on the same machine. The isolated phase shows the flow-control change most clearly: it commits about 5,000 transactions a second on one thread, so with the journal written every 5 s up to ~25,000 commits wait in the queue. Under the old limit of 4,096, many of those commits were forced to flush the journal to disk. A run's batch of ~25,000 small changesets is only a few tens of MB, well below the new 128 MB limit.

rws run	mean, previous	mean, now	change	p99.99, previous	p99.99, now
Insert, isolated	251 µs	207 µs	-18%	21 ms	3.46 ms
Find by ID, isolated	41 µs	38 µs	-8%	1.09 ms	1.09 ms
Find earliest expiry, isolated	28 µs	27 µs	-5%	806 µs	127 µs
Delete by ID, isolated	247 µs	230 µs	-7%	16 ms	2.03 ms
Delete by expiry, isolated	219 µs	208 µs	-5%	16 ms	1.97 ms
State transition, isolated	408 µs	381 µs	-7%	21 ms	8.84 ms
Recycle storage, isolated	403 µs	378 µs	-6%	19 ms	3.13 ms

Figure 1. p99.99 latency per transaction, isolated, rws run

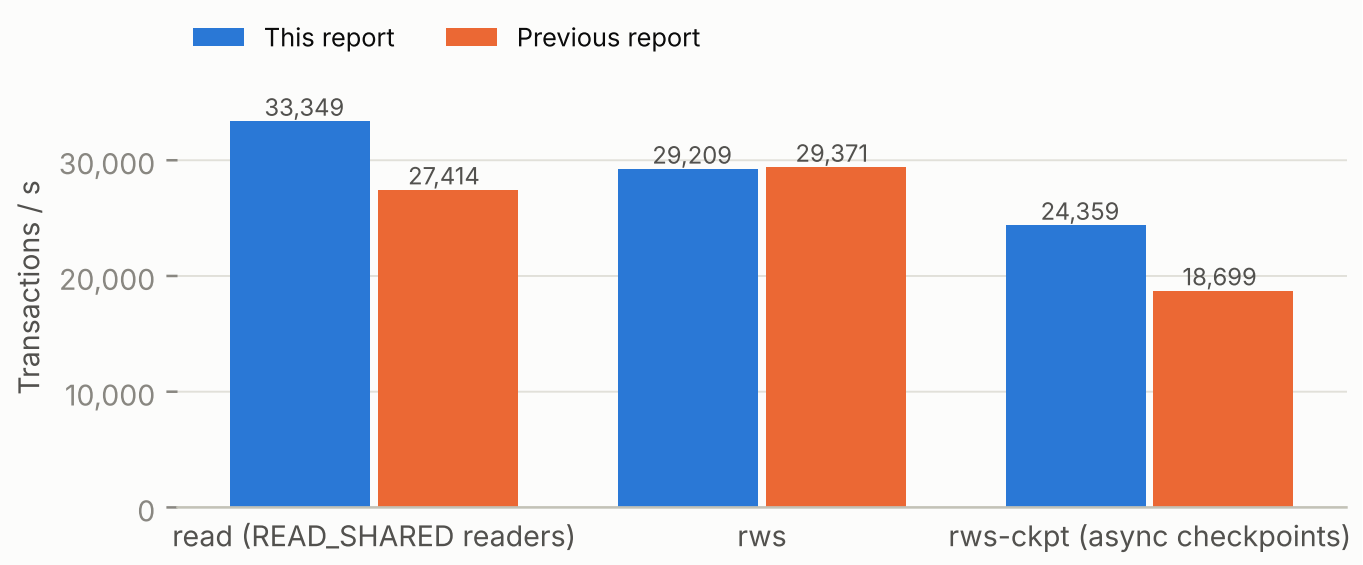
Logarithmic scale. This report against the previous report (flow control at 4,096 queued commits).



Mixed phase, all transactions	txn/s, previous	txn/s, now	change	p99.99, previous	p99.99, now	max, previous	max, now
read	27,414	33,349	+22%	41 ms	3.55 ms	254 ms	7.88 ms
rws	29,371	29,209	-1%	23 ms	2.66 ms	125 ms	19 ms
rws-ckpt	18,699	24,359	+30%	50 ms	41 ms	1.35 s	111 ms

Figure 2. Mixed-workload throughput, this report against the previous

Transactions per second over the 30 s measurement.



Isolated operations

Each operation runs as a complete transaction on one thread, with nothing else running. Inserts and deletes take 204–231 µs (about 450,000–500,000 cycles), and the in-place updates (state transition, recycle) 370–387 µs (810,000–840,000 cycles). Reads take 26–38 µs (58,000–83,000 cycles). The three runs agree within 5% on every mean (see Appendix).

Write tails are now 1.3–8.8 ms at p99.99 rather than 14–22 ms. They're still longer than read tails, and vary more between runs, so there is still something in the write path worth profiling, but it's no longer the dominant effect.

Mixed workload

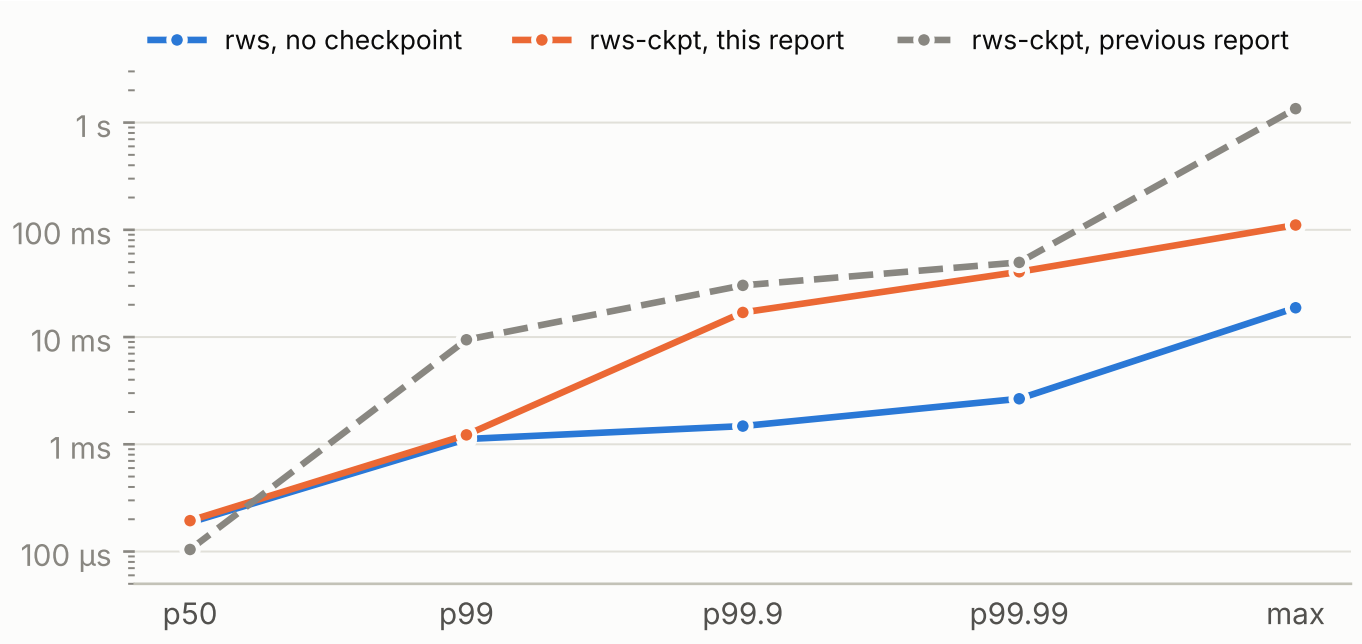
READ_SHARED and READ_WRITE_SHARED transactions take the database's read/write lock in different modes, so they run in alternating batches. With readers in READ_SHARED, reads dominate: 29,293 reads/s against 4,056 writes/s. With every transaction in READ_WRITE_SHARED, writers nearly double to 7,471/s and the total is slightly lower. The medians show the same trade: a find by ID takes 67 µs with READ_SHARED readers against 168 µs, while a delete by ID waits longer (1.57 ms against 552 µs). Total CPU per transaction was 199,018 cycles (read run) and 374,922 (rws run). No transaction hit a concurrency clash in any run.

Asynchronous checkpoints

The rws-ckpt run repeats the rws run with a separate thread writing asynchronous checkpoints back to back during the mixed phase. 4 checkpoints ran, and one was running 92% of the measured time. The comparison is between the two runs.

Figure 3. Latency percentiles, all mixed-phase transactions

Logarithmic scale. rws and rws-ckpt from this report; rws-ckpt from the previous report, dashed.



Measure	rws (no checkpoint)	rws-ckpt	Change	Change in previous report
Throughput (txn/s)	29,209	24,359	-17%	-36%
Writer transactions/s	7,471	6,316	-15%	-55%
Inserts/s	3,076	2,585	-16%	-55%
p50	187 µs	194 µs	+4%	-40%
p99	1.12 ms	1.22 ms	×1.1	×8.0
p99.9	1.48 ms	17 ms	×11	×18
p99.99	2.66 ms	41 ms	×15	×2.1
Max	19 ms	111 ms	×5.9	×11
Process CPU cycles per transaction	374,922	515,993	+38%	+87%

Up to p99 a checkpoint now makes little difference (1.12 ms → 1.22 ms). Beyond that the tail still grows: p99.9 to 17 ms, p99.99 to 41 ms, worst case 111 ms. Those percentiles are still lower than in the previous report, and the 1.35 s worst case is gone.

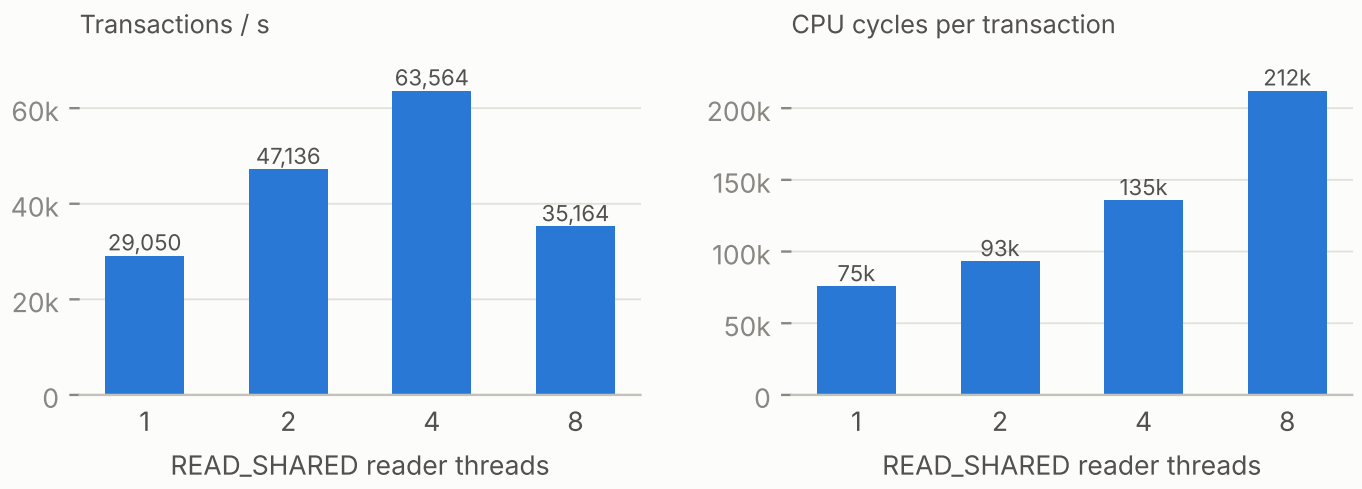
Checkpoints themselves got faster: the 3 timed during the measurement took 9.6 s on average, against 14.2 s before, and about 3.8 times a synchronous checkpoint of the same data (2.5 s). The checkpoint thread used 48.4 billion cycles in the 31.5 s measured, about 22 s of CPU, so the checkpoint is largely CPU-bound.

Read-only workload

Readers only, no writers, against a freshly reopened database. Every find hits a live row (no misses), unlike the mixed runs.

Figure 4. Read-only throughput and CPU cost by number of readers

READ_SHARED readers. Left: transactions per second. Right: CPU cycles per transaction, whole process.

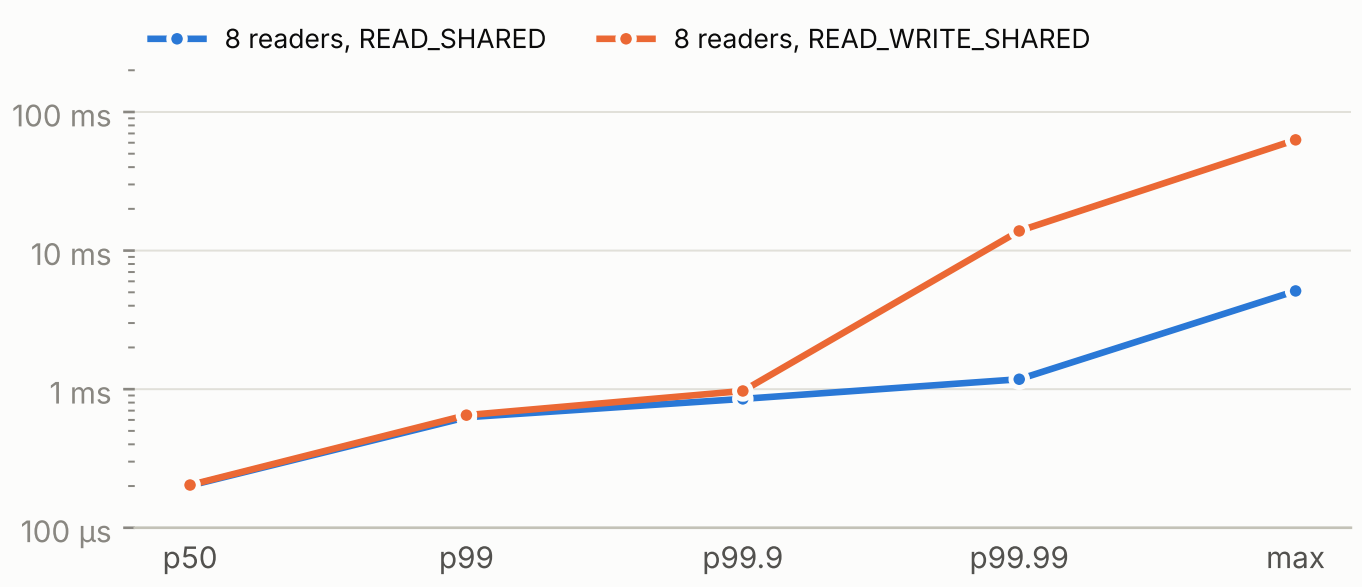


Readers	txn/s	vs 1 reader	mean	p50	p99	p99.9	p99.99	max	cycles per txn
1, READ_SHARED	29,050	×1.00	34 µs	34 µs	63 µs	107 µs	306 µs	4.99 ms	75k
2, READ_SHARED	47,136	×1.62	42 µs	42 µs	71 µs	127 µs	250 µs	2.69 ms	93k
4, READ_SHARED	63,564	×2.19	63 µs	57 µs	121 µs	166 µs	265 µs	7.03 ms	135k
8, READ_SHARED	35,164	×1.21	227 µs	202 µs	627 µs	851 µs	1.18 ms	5.11 ms	212k
8, READ_WRITE_SHARED	34,426	×1.19	232 µs	203 µs	649 µs	970 µs	14 ms	63 ms	215k

One reader runs 29,050 txn/s, a find by ID taking 35 µs, matching the isolated phase. Throughput scales to 2 and 4 readers, though sub-linearly (×1.62 and ×2.19), and falls back to 35,164 with 8. CPU per transaction rises with every added reader, almost three times over from 1 reader to 8, while the work per transaction is the same. That pattern points to contention on a shared lock rather than a shortage of CPU. Candidates are the C API's handle table, which has a single lock taken on every call, and MemDB2's own database lock. With 8 threads on 4 cores, a thread holding a lock is also more likely to be descheduled while others wait for it.

Figure 5. Latency percentiles, 8 readers, READ_SHARED against READ_WRITE_SHARED

Logarithmic scale. All transactions, no writers.



With 8 READ_WRITE_SHARED readers, throughput and median match READ_SHARED, but the tail jumps to 14 ms at p99.99. No row changes and every transaction rolls back, so this tail comes from beginning or ending a READ_WRITE_SHARED transaction, not from the journal. A delay of ~14 ms is close to Windows' default 15.6 ms timer tick, which suggests a timed wait or sleep somewhere in that path. The same ~14 ms appears at p99.99 for writes in the rws mixed run.

Loading and startup

The 1M rows loaded in 61 s–62 s with 8 threads (16,172–16,421 rows/s), against 125 s–144 s single-threaded in the previous report. No batch hit a concurrency clash. The table used 1,906 bytes of process memory per row against 1,623 before; the cause isn't measured here. A synchronous checkpoint of the loaded data took 2.5 s, as before.

Reopening a copy of the checkpointed database took 30.5 s–31.8 s, about half the parallel load time, and counting its rows in storage order took about 2.3 s. Neither is part of any measured figure.

Caveats

- **Old hardware.** The tests ran on a 2012 Ivy Bridge laptop (Core i7-3632QM, 2.2 GHz, 4 cores with Hyper-Threading). Absolute figures are unlikely to be representative of modern hardware; the comparisons between configurations are more meaningful than the raw numbers.
- **Every call goes through the C DLL shim.** Each transaction makes about 6–10 C API calls. Each call validates and looks up its handle in a handle table guarded by a single lock, and fields are named by string on every read and write. The shim is likely to be considerably slower than using MemDB2 natively from Delphi, and its lock may limit the read-only scaling seen here.
- **Different starting state from the previous report.** This report's runs start from a database loaded by 8 threads and reopened from its checkpoint; the previous report's started from a freshly loaded, single-threaded table. Row order and memory layout differ, which may account for part of the changes besides flow control.
- **Expiry processing wasn't exercised in the mixed phase.** Records expire 1,386,294 inserts after their own insert, but only 65,000–124,000 ids were issued in each mixed phase, so nothing expired, and every delete-by-expiry and recycle transaction there was a miss. In the isolated phase those transactions did real work.
- **Misses are included in the latencies.** About 43% of mixed-phase finds by ID, and a similar share of deletes by ID, targeted rows already deleted.
- **Checkpoint overlap.** Checkpoints ran back to back, so the benchmark counted every rws-ckpt transaction as overlapping one. Treat its figures as an average over the whole phase.
- **One run per configuration.** The isolated phase is the same in all three mixed runs and agrees within 5%, which gives a rough idea of run-to-run variation.
- **Not covered here:** the baseline engine, flush-per-commit journalling, x86, the specified 5 million rows, a database larger than RAM, and allocations per transaction.

Appendix: data

Isolated phase, read run

Operation	txn/s	mean	p50	p99	p99.9	p99.99	max	thread cycles	process cycles
Insert	4,900	206 µs	200 µs	336 µs	731 µs	2.72 ms	9.01 ms	449k	453k
Find by ID	27,500	36 µs	35 µs	67 µs	125 µs	399 µs	1.43 ms	80k	80k
Find earliest expiry	37,100	27 µs	26 µs	49 µs	113 µs	392 µs	1.01 ms	59k	59k
Delete by ID	4,400	229 µs	222 µs	358 µs	776 µs	3.31 ms	33 ms	498k	504k
Delete by expiry	4,800	206 µs	200 µs	340 µs	761 µs	3.05 ms	10 ms	450k	455k
State transition	2,700	373 µs	362 µs	560 µs	1.28 ms	4.54 ms	11 ms	814k	821k
Recycle storage	2,600	387 µs	369 µs	687 µs	2.21 ms	7.05 ms	16 ms	841k	847k

Isolated phase, rws run

Operation	txn/s	mean	p50	p99	p99.9	p99.99	max	thread cycles	process cycles
Insert	4,800	207 µs	200 µs	336 µs	657 µs	3.46 ms	8.62 ms	451k	456k
Find by ID	26,300	38 µs	36 µs	71 µs	157 µs	1.09 ms	2.18 ms	83k	83k
Find earliest expiry	37,500	27 µs	26 µs	48 µs	76 µs	127 µs	221 µs	58k	58k
Delete by ID	4,300	230 µs	226 µs	332 µs	530 µs	2.03 ms	11 ms	504k	509k
Delete by expiry	4,800	208 µs	203 µs	321 µs	522 µs	1.97 ms	11 ms	455k	460k
State transition	2,600	381 µs	369 µs	627 µs	1.37 ms	8.84 ms	33 ms	828k	836k
Recycle storage	2,600	378 µs	369 µs	522 µs	873 µs	3.13 ms	7.84 ms	828k	834k

Isolated phase, rws-ckpt run (no checkpoints in this phase)

Operation	txn/s	mean	p50	p99	p99.9	p99.99	max	thread cycles	process cycles
Insert	4,900	204 µs	200 µs	299 µs	493 µs	2.09 ms	7.96 ms	446k	450k
Find by ID	27,600	36 µs	35 µs	64 µs	123 µs	280 µs	1.75 ms	79k	80k
Find earliest expiry	37,400	27 µs	26 µs	47 µs	82 µs	159 µs	1.12 ms	58k	58k
Delete by ID	4,400	227 µs	222 µs	332 µs	522 µs	1.66 ms	11 ms	495k	501k
Delete by expiry	4,900	205 µs	200 µs	302 µs	500 µs	1.28 ms	12 ms	448k	453k
State transition	2,700	370 µs	362 µs	515 µs	896 µs	3.94 ms	9.73 ms	808k	815k
Recycle storage	2,600	381 µs	369 µs	597 µs	1.07 ms	5.20 ms	8.78 ms	832k	839k

Cycles are CPU cycles per transaction: thread = the calling thread; process = the whole process, including MemDB2's journal thread.

Mixed phase, read run (readers READ_SHARED)

Operation	txns	txn/s	mean	p50	p99	p99.9	p99.99	max	misses
Insert	51,700	1,700	949 µs	612 µs	2.60 ms	3.28 ms	6.93 ms	7.38 ms	0
Find by ID	823,618	26,400	137 µs	67 µs	1.30 ms	1.61 ms	2.27 ms	7.03 ms	352,686
Find earliest expiry	91,513	2,900	128 µs	58 µs	1.28 ms	1.58 ms	2.21 ms	6.98 ms	0
Delete by ID	25,854	800	1.35 ms	1.57 ms	2.57 ms	3.25 ms	7.52 ms	7.69 ms	11,076
Delete by expiry	31,068	1,000	673 µs	63 µs	2.09 ms	2.81 ms	3.46 ms	4.98 ms	31,068
State transition	12,880	400	1.20 ms	1.28 ms	2.84 ms	3.79 ms	7.76 ms	7.88 ms	5,538
Recycle storage	5,201	200	846 µs	1.21 ms	2.15 ms	2.84 ms	4.23 ms	4.23 ms	5,201

Mixed phase, rws run (readers READ_WRITE_SHARED)

Operation	txns	txn/s	mean	p50	p99	p99.9	p99.99	max	misses
Insert	96,259	3,100	743 µs	724 µs	1.34 ms	1.85 ms	14 ms	19 ms	0
Find by ID	612,236	19,600	185 µs	168 µs	493 µs	694 µs	1.13 ms	18 ms	260,751
Find earliest expiry	68,026	2,200	175 µs	159 µs	485 µs	702 µs	1.00 ms	18 ms	0
Delete by ID	48,129	1,500	535 µs	552 µs	1.30 ms	1.81 ms	15 ms	17 ms	20,595
Delete by expiry	55,752	1,800	188 µs	164 µs	560 µs	821 µs	1.13 ms	18 ms	55,752
State transition	23,961	800	643 µs	754 µs	1.48 ms	2.00 ms	14 ms	16 ms	10,144
Recycle storage	9,688	300	193 µs	168 µs	575 µs	769 µs	2.20 ms	2.20 ms	9,688

Mixed phase, rws-ckpt run (async checkpoints running)

Operation	txns	txn/s	mean	p50	p99	p99.9	p99.99	max	misses
Insert	81,421	2,600	876 µs	739 µs	2.33 ms	27 ms	64 ms	111 ms	0
Find by ID	511,381	16,200	223 µs	174 µs	567 µs	12 ms	33 ms	69 ms	218,184
Find earliest expiry	56,821	1,800	211 µs	168 µs	560 µs	10 ms	33 ms	50 ms	0
Delete by ID	40,716	1,300	640 µs	567 µs	1.58 ms	25 ms	59 ms	69 ms	17,410
Delete by expiry	48,338	1,500	232 µs	177 µs	649 µs	9.67 ms	43 ms	70 ms	48,338
State transition	20,282	600	764 µs	746 µs	1.84 ms	29 ms	62 ms	68 ms	8,663
Recycle storage	8,150	300	216 µs	179 µs	642 µs	2.09 ms	24 ms	24 ms	8,150

Read-only phase, 8 READ_SHARED readers

Operation	txns	txn/s	mean	p50	p99	p99.9	p99.99	max	misses
Find by ID	1,975,967	31,600	231 µs	205 µs	634 µs	858 µs	1.18 ms	5.11 ms	0
Find earliest expiry	219,549	3,500	194 µs	179 µs	590 µs	814 µs	1.30 ms	4.97 ms	0

Read-only phase, 8 READ_WRITE_SHARED readers

Operation	txns	txn/s	mean	p50	p99	p99.9	p99.99	max	misses
Find by ID	1,938,164	31,000	236 µs	207 µs	649 µs	970 µs	14 ms	63 ms	0
Find earliest expiry	215,352	3,400	200 µs	179 µs	612 µs	955 µs	14 ms	53 ms	0

Source: `Common\MemDB2\Bench\results.csv`, `results_execution.log`, `results_readonly.csv` and `results_readonly_execution.log`, 12 November 2026. Previous figures: MemDB2_Benchmark_Results_2026-11-11.pdf.